ADF Incremental Load Implementation & Troubleshooting Guide

Automotive Sales & Finance Data Engineering Platform

PostgreSQL → Azure Data Factory → ADLS Gen2 Bronze

Purpose

This guide documents the incremental-loading design implemented in the project, the row-mismatch issue and fixes, ADF activity expressions, PostgreSQL SQL, test evidence, and practical cautions for future runs.

Current result

Check	Observed result	Interpretation
Copy activity	1 row read; 1 row copied	Test order passed the incremental filter and Copy activity reported a successful write.
Lookup_max_created	2026-10-10T12:05:00Z	ADF detected the latest qualifying timestamp.
Watermark after run	2026-10-10 12:05:00	The checkpoint advanced automatically.

Important scope note: The successful test used deliberately inserted record 7004 (ORD-TEST-7004), with created_at = 2026-10-10 12:05:00. This validates the incremental path for that test record; it does not by itself prove that every historical record is present exactly once across the initial snapshot and all Bronze batches.

Preserve **orders_initial_backup.csv**. Do not overwrite or delete the original snapshot.

1. What incremental loading means

A full load copies every source row on each run. Incremental loading copies only rows that qualify as newer than a saved checkpoint, or watermark. In this implementation, the checkpoint is a timestamp in public.etl_watermark and the source filter uses sales.orders.created_at.

```
SELECT *
FROM sales.orders
WHERE created_at > TIMESTAMP '2026-10-10 12:00:00'
ORDER BY created_at;
```

The timestamp shown above is an example from the test. In ADF, the cutoff is dynamically taken from the watermark Lookup output.

2. Architecture and activity sequence

```
PostgreSQL: sales.orders
        |
        v
Lookup_Last_Watermark
        |
        v
Copy_PostgreSQL_To_Bronze  ---> ADLS Gen2 / bronze / sales / unique batch file
        |
        v
Lookup_max_created
        |
        v
If Condition1: max_created_at is not null?
        | True                    | False
        v                         v
```

```
Lookup_Update_Watermark       No update
```

The intended order is: read the checkpoint → copy qualifying rows → find the maximum created_at among qualifying rows → update the checkpoint only if a qualifying timestamp exists.

Activity names in this document reflect the names used in the project. The reusable pipeline is PL_02_PostgreSQL_To_Bronze_Reusable and the watermark key is PL_02_sales_orders_incremental.

3. What went wrong and what was fixed

3.1 Old hardcoded query ran at runtime

Monitor initially showed a fixed April timestamp even though a dynamic expression was visible in Author. This meant the actual run still used the old query.

Fix: Applied the expression through Source → Query → Add dynamic content, clicked OK, saved, then inspected the newest Monitor run's Copy Input. Runtime query later showed the current watermark value.

3.2 Wrong reusable dataset parameters

A watermark Lookup run input initially showed schema_name=customer and table_name=customers (including a trailing space), rather than the intended watermark table.

Fix: Set activity-level dataset parameters explicitly: public / etl_watermark for watermark activities, and sales / orders for Lookup_max_created.

3.3 Row-count comparisons were not like-for-like

A single incremental batch contains only rows newer than its starting watermark. Its row count should not be compared directly with the entire historical source table.

Fix: Compare the source query using the same pre-run watermark with ADF rowsRead and rowsCopied, then inspect the output file. Historical reconciliation must combine the initial snapshot and all batches.

3.4 Empty batches

When no records qualified, Lookup_max_created returned null.

Fix: If Condition1 checks whether max_created_at is not null. Lookup_Update_Watermark is inside the True branch; the False branch is empty.

3.5 Watermark state

The watermark must be verified in PostgreSQL; an expression in Author alone is not proof of the persisted checkpoint.

Fix: Queried public.etl_watermark after the run and confirmed it advanced to 2026-10-10 12:05:00.

4. ADF activities and exact expressions

4.1 Lookup_Last_Watermark — Query

Dataset: DS_PostgreSQL_Customer. Activity parameters: schema_name=public, table_name=etl_watermark.

```
SELECT last_loaded_timestamp
FROM public.etl_watermark
WHERE pipeline_name = 'PL_02_sales_orders_incremental';
```

Expected output shape:

```
{
  "firstRow": {
    "last_loaded_timestamp": "2026-10-10T12:00:00Z"
  }
}
```

The timestamp is dynamic and changes after successful batches.

4.2 Copy_PostgreSQL_To_Bronze — Source → Query → Add dynamic content

```
@concat(
    'SELECT * FROM sales.orders WHERE created_at > CAST(''',
    activity('Lookup_Last_Watermark').output.firstRow.last_loaded_timestamp,
    ''' AS TIMESTAMP) ORDER BY created_at'
)
```

Example runtime SQL observed in Monitor:

```
SELECT * FROM sales.orders
WHERE created_at > CAST('2026-10-10T12:00:00' AS TIMESTAMP)
ORDER BY created_at
```

Verification: Monitor → newest run → Copy_PostgreSQL_To_Bronze → Input → source.query. Check runtime SQL, not just the Author editor.

4.3 Lookup_max_created — Query dynamic content

Dataset parameters: schema_name=sales, table_name=orders.

```
@concat(
    'SELECT MAX(created_at) AS max_created_at ',
    'FROM sales.orders ',
    'WHERE created_at > CAST(''',
    activity('Lookup_Last_Watermark').output.firstRow.last_loaded_timestamp,
    ''' AS TIMESTAMP)'
)
```

Test output:

```
{
  "firstRow": {
    "max_created_at": "2026-10-10T12:05:00Z"
  }
}
```

If no rows qualify, MAX(created_at) returns null.

4.4 If Condition1 — expression

```
@not(equals(activity('Lookup_max_created').output.firstRow.max_created_at, null))
```

True branch: Lookup_Update_Watermark. False branch: empty. This prevents a checkpoint update for an empty batch.

4.5 Lookup_Update_Watermark — Query dynamic content

Dataset parameters: schema_name=public, table_name=etl_watermark.

```
@concat(
    'SELECT public.update_etl_watermark_fn(''PL_02_sales_orders_incremental'', CAST(''',
    activity('Lookup_max_created').output.firstRow.max_created_at,
    ''' AS TIMESTAMP)) AS watermark_updated'
)
```

The configured PostgreSQL wrapper function calls public.update_etl_watermark. The successful test showed the stored watermark advancing to 12:05:00. If a future run fails, inspect that run's actual Input, Output, and status.

4.6 Sink file naming

The sink writes to container bronze and directory sales via dataset/pipeline parameters. A unique run-ID-based filename was configured:

```
@concat(
    pipeline().parameters.table_name,
    '_',
    pipeline().RunId,
    '.csv'
)
```

This helps avoid overwriting a prior run's batch file. It does not by itself prevent duplicate logical records if the pipeline is rerun.

5. PostgreSQL setup SQL

5.1 Create the watermark table

```
CREATE TABLE IF NOT EXISTS public.etl_watermark (
    pipeline_name VARCHAR(100) PRIMARY KEY,
    last_loaded_timestamp TIMESTAMP
);
```

5.2 Insert the initial watermark row — setup example only

```
INSERT INTO public.etl_watermark (
    pipeline_name,
    last_loaded_timestamp
)
VALUES (
    'PL_02_sales_orders_incremental',
    TIMESTAMP '2026-04-25 07:52:18.037335'
);
```

Do not run this INSERT again blindly: the row already exists. The timestamp above is the historical initial setup example, not the current watermark.

5.3 Stored procedure to update or insert the watermark

```
CREATE OR REPLACE PROCEDURE public.update_etl_watermark(
    p_pipeline_name VARCHAR(100),
    p_last_loaded_timestamp TIMESTAMP
)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE public.etl_watermark
    SET last_loaded_timestamp = p_last_loaded_timestamp
    WHERE pipeline_name = p_pipeline_name;

    IF NOT FOUND THEN
        INSERT INTO public.etl_watermark (
            pipeline_name,
            last_loaded_timestamp
        )
        VALUES (
            p_pipeline_name,
            p_last_loaded_timestamp
        );
    END IF;
END;
$$;
```

5.4 Function wrapper used by the ADF Lookup query

```
CREATE OR REPLACE FUNCTION public.update_etl_watermark_fn(
    p_pipeline_name VARCHAR(100),
    p_last_loaded_timestamp TIMESTAMP
)
RETURNS BOOLEAN
LANGUAGE plpgsql
AS $$
BEGIN
    CALL public.update_etl_watermark(
        p_pipeline_name,
        p_last_loaded_timestamp
    );
    RETURN TRUE;
END;
$$;
```

5.5 Manually call the procedure — only for controlled manual testing

```
CALL public.update_etl_watermark(
    'PL_02_sales_orders_incremental',
    TIMESTAMP '2026-10-10 12:05:00'
);
```

Do not manually advance the production watermark just to test; doing so can cause records to be skipped.

6. Manual SQL validation queries

6.1 Check the persisted watermark

```
SELECT pipeline_name, last_loaded_timestamp
FROM public.etl_watermark
WHERE pipeline_name = 'PL_02_sales_orders_incremental';
```

Latest verified value: 2026-10-10 12:05:00.

6.2 Check records newer than the current watermark

```
SELECT order_id, order_number, created_at
FROM sales.orders
WHERE created_at > (
    SELECT last_loaded_timestamp
    FROM public.etl_watermark
    WHERE pipeline_name = 'PL_02_sales_orders_incremental'
)
ORDER BY created_at, order_id;
```

After the watermark reaches 12:05:00, this should return zero rows if no later records exist.

6.3 Verify test record 7004

```
SELECT order_id, order_number, created_at
FROM sales.orders
WHERE order_id = 7004;
```

Expected: ORD-TEST-7004, created_at 2026-10-10 12:05:00.

6.4 Recreate the test batch predicate using the prior watermark

```
SELECT order_id, order_number, created_at
FROM sales.orders
WHERE created_at > TIMESTAMP '2026-10-10 12:00:00'
ORDER BY created_at, order_id;
```

This helps identify the source rows eligible for the tested batch. Use the actual starting watermark for each batch being reconciled.

6.5 Calculate the maximum qualifying timestamp

```
SELECT MAX(created_at) AS max_created_at
FROM sales.orders
WHERE created_at > TIMESTAMP '2026-10-10 12:00:00';
```

6.6 Count all source orders

```
SELECT COUNT(*) AS total_orders
FROM sales.orders;
```

This total is useful for source profiling, but do not compare it directly to a single incremental batch count.

6.7 Find duplicate order IDs

```
SELECT order_id, COUNT(*) AS row_count
FROM sales.orders
GROUP BY order_id
HAVING COUNT(*) > 1
ORDER BY order_id;
```

If order_id is a primary key, no rows should be returned.

6.8 Inspect the latest source orders

```
SELECT order_id, order_number, created_at
FROM sales.orders
ORDER BY created_at DESC
LIMIT 20;
```

6.9 Inspect every watermark entry

```
SELECT *
FROM public.etl_watermark
```

```
ORDER BY pipeline_name;
```

7. Test record used and test results

The deliberate test record used to verify the pipeline:

```
INSERT INTO sales.orders (
    order_id, order_number, order_date, order_status,
    total_amount, discount_amount, tax_amount, net_amount, created_at
)
VALUES (
    7004, 'ORD-TEST-7004', DATE '2026-10-10', 'Confirmed',
    50000.00, 0.00, 9000.00, 59000.00,
    TIMESTAMP '2026-10-10 12:05:00'
);
```

This INSERT has already been executed. Do not execute it again. The starting watermark was 12:00:00, so order 7004 qualified.

Check	Result
Copy activity	1 row read, 1 row copied
Lookup_max_created	2026-10-10T12:05:00Z
Watermark after run	2026-10-10 12:05:00

8. Important loopholes and production cautions

• Timestamp ties: created_at > watermark can skip rows that share the exact checkpoint timestamp if they were not all copied. Consider a composite watermark (created_at, order_id) or an overlap window plus downstream deduplication.

• Updates: created_at does not capture changes to existing rows. Use a reliable updated_at column or CDC if updates must be propagated.

• Deletes: hard deletes are invisible to a timestamp filter unless tracked separately.

• Retries: unique run-ID filenames prevent overwriting files but reruns can still create duplicate logical records. Track batch status and deduplicate downstream.

• Watermark timing: advance it only after Copy and validation succeed.

• Concurrent runs: two runs can read the same watermark and duplicate work. Prevent overlapping runs for the same watermark.

• Timezone consistency: verify timestamp interpretation between PostgreSQL and ADF; the Lookup output may display a Z suffix.

• Initial snapshot: keep orders_initial_backup.csv untouched. Reconcile the initial snapshot plus all batches, not one batch against the whole source.

• Connector caveat: the configured Lookup calls a side-effecting PostgreSQL function. If this behavior fails in a future connector/runtime version, inspect actual activity Input/Output and switch to a supported update mechanism.

9. Troubleshooting checklist

• Confirm the latest Monitor run is PL_02_PostgreSQL_To_Bronze_Reusable.

• Check Lookup_Last_Watermark output and its dataset parameters (public / etl_watermark).

• Check Copy Input → source.query for the evaluated runtime timestamp.

• Compare rowsRead and rowsCopied against the same source predicate and starting watermark.

• Check Lookup_max_created output; null means no qualifying records.

• Confirm Lookup_Update_Watermark status and inspect its Input/Output.

• Run the manual watermark SELECT in PostgreSQL after the run.

• Inspect the generated Bronze file and verify expected rows.

• Preserve orders_initial_backup.csv.

• Before production, address timestamp ties, retries, updates/deletes, and overlapping runs.

10. Quick revision

Remember the flow: Read watermark → Copy newer rows → Find maximum qualifying timestamp → If not null, update watermark → Verify in PostgreSQL.

What was fixed: the Copy activity had been running an old hardcoded timestamp; the dynamic expression was applied and verified in Monitor. The reusable watermark Lookup dataset parameters were corrected. Empty-batch handling was configured. The test record was copied and the watermark advanced.

What remains: independently reconcile the original snapshot plus all batch files and harden the pipeline for timestamp ties, retries, updates, deletes, and concurrent runs.